HW 4.3

Charlie Coyle

2026-04-14

Busiest Airports Analysis

#Passenger Traffic Table

Table 1: Table 1: Passenger traffic for six of the busiest airports in the world from 2020 to 2025.

Airport Name	IATA Code	2020	2021	2022	2023	2024	2025
Beijing Daxing International	PKX	25051012	49441029	53618949	NA	NA	NA
Frankfurt	FRA	24812849	61564957	63189666	18768601	48918482	59355389
Hamad International	DOH	NA	52700000	54300000	NA	35730482	45916098
John F. Kennedy International	JFK	30788322	63265984	62629455	16630642	55287693	62464331
Miami International	MIA	37302456	55926566	55314661	18663858	50684396	52340934

Passenger Traffic visulization

Overall from the table and the plot we see that for every airport there is an increase in passenger traffic over the years, with the growth being more rapid right after 2020 and eventually plateauing more and more as the years go on. This can definitely be attributed to the world going back to normal after the Covid pandemic, as when airports fully opened back up right away there was a huge spike up in people traveling and flying. The atlanta airport is far an away the busiest and stayed that way over all the years shown in the chart, so we can assume that it could be the largest, offers the most flights, or even that it is a large connecting hub. We can also assume that next year there will be a similar amount of air traffic as last year as from 24 to 25 none of the airports had a signifigant increase.

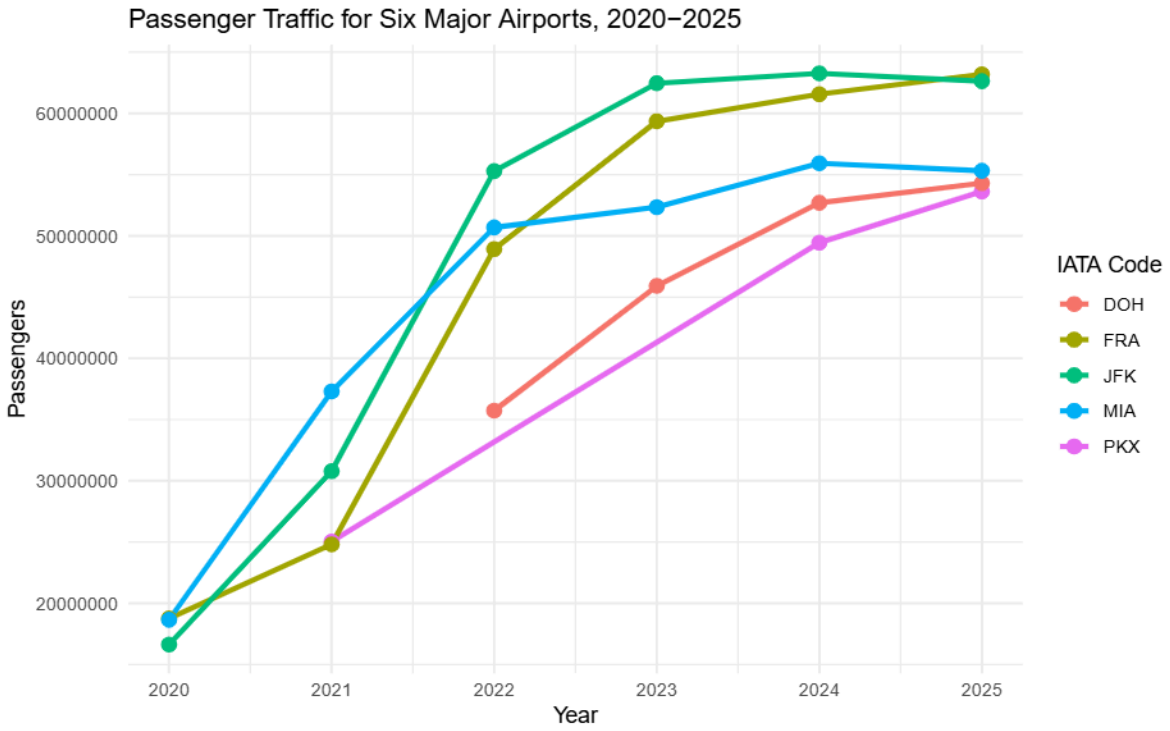
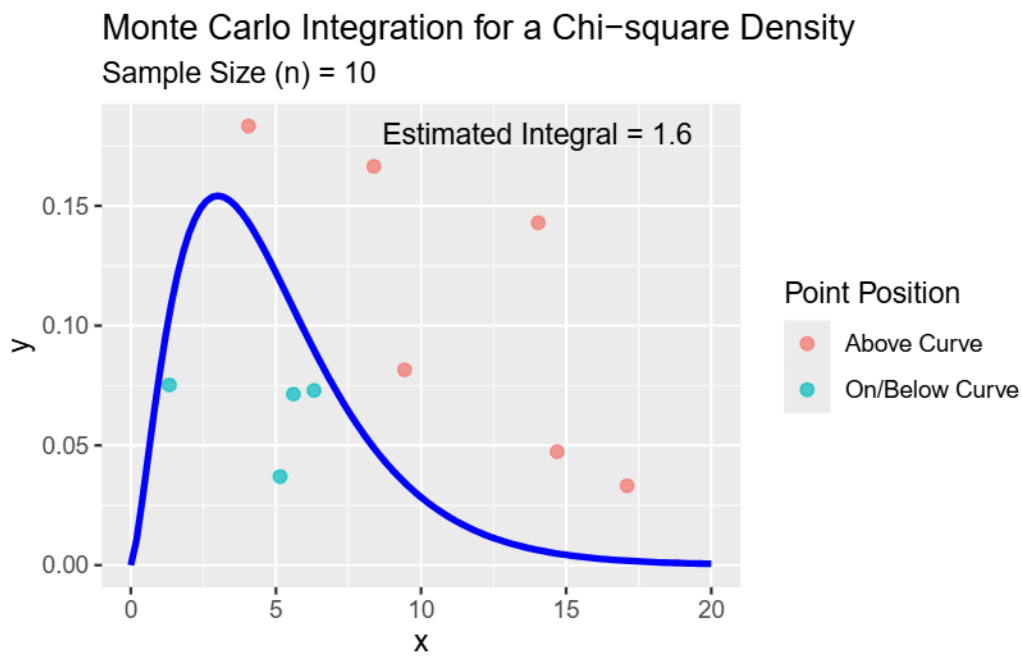
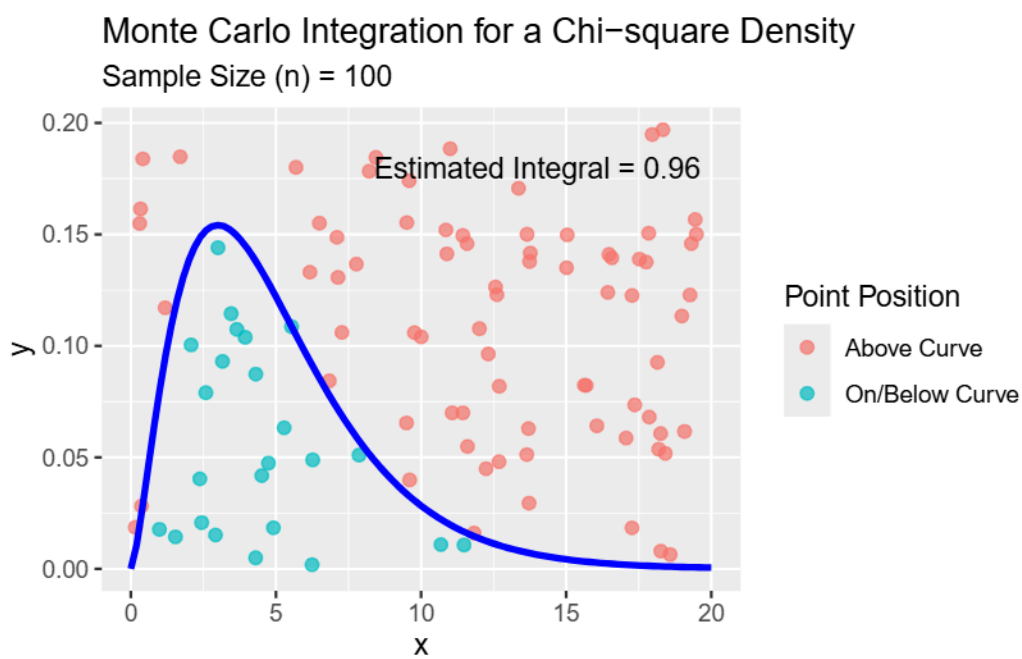
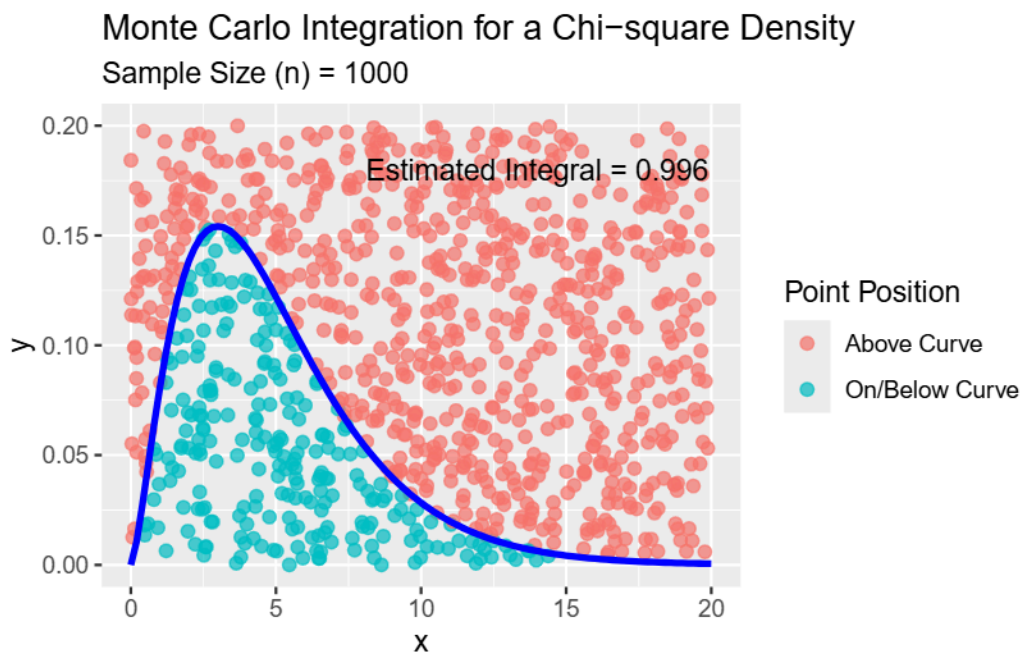


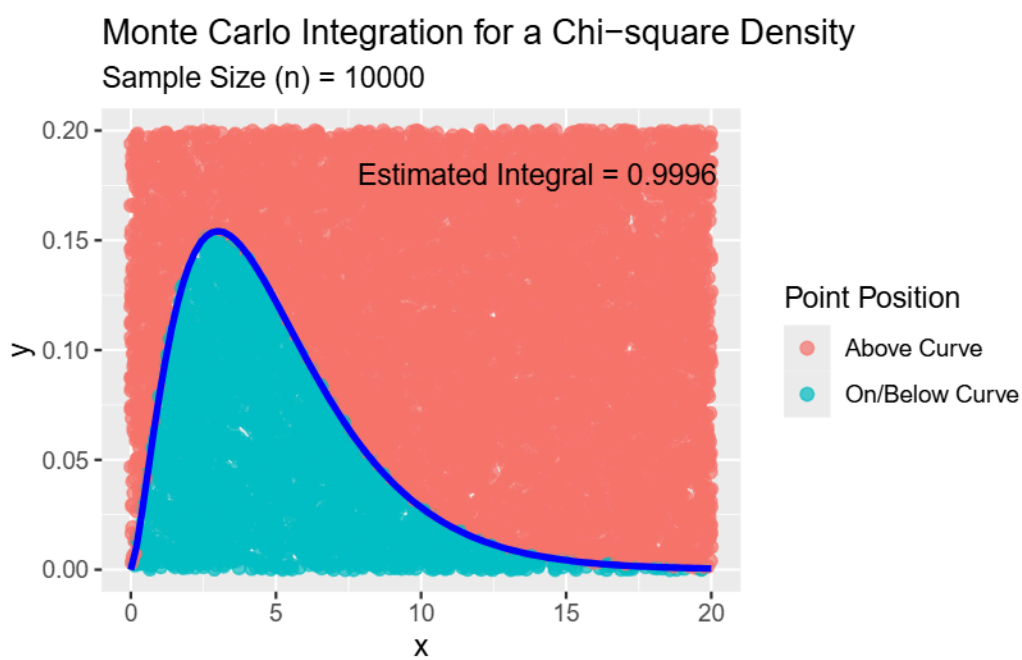
Figure 1: Figure 1: Passenger traffic for six major airports from 2020 to 2025.

Monte Carlo simulation plots









The more random points that are in the sample, the accuracy as well as the stability of the MC estimate increase dramatically. However once it gets high enough, it starts to be more consistent even with more values added. We see this when increasing from 1000 to 10000, and we can assume if we went even higher we would see even less of a difference. Since it is a valid

prob distribution, we can assume that the exact value over the full range of the integral is 1.

#Calcium Data Plan

Goal:

Tidy and wrangle the calcuim.csv dataset and then produce a data visulazation so that someone can understand or analyze changes in calcium levels overtime as well as see differences between groups

Needs: I will need the ggplot2 package as it has the commands for visualization, the tidyverse package for wrangling, and the readr package to load the csv. I will also need to restrucre the data set since it is not tidy and I will tidy it so that each row is a single observation, and there are columns for treatment group, year, and measurement. As for additional verbs in order to reshape the data I will use the pivot longer function. In terms of visualization, line plot geometery will be the best to see the changes in calcium levels for each group.

Steps: 1: create my plan 2: create a blank R script 3: Copy and paste the wrangling template 4: fill in the wrangling template w/ context 5: start writing code 6: Load the packages and data 7: Inspect the dataset and view structre 8: Create a subject ID varible 9: reshape the data using pivot_longer() 10: Create a treatment group variable 11: create a time variable 12:Clean/rename varibales 13: Define the framework and mapping for the plot/visulization 14: Specify the line geometery in order to show changes over time 15: Label title axes and legend 16: render the plot 17: debug errors 18: Create a list of improvements and evaluate plot, then update plan accordingly.

#prompt for AI Using the calcium.csv dataset follow my plan to tidy and wrangle the data. The dataset is not tidy and contains calcium measurements across time for different two groups Reshape the data into a tidy format with columns for subject ID, treatment group, time (Initial, Year 1, Year 2, Year 3), and calcium measurement. Then create a ggplot2 visualization that shows how calcium levels change over time for both groups. Use a line plot and include labels. Then summarize and explain what the plot shows about differences between the two groups.

#AI output with my prompt

See Ai code appendix

#Generic AI output

Generic output

I created a visualization showing the average value of each column in your dataset. What this shows: • Each point represents the mean (average) of one column (y0, y1, y2, y3, etc.). • You can quickly compare which variables tend to have higher or lower values overall. • For example, some columns cluster around ~70–73, while others drop closer to ~65.

If you want something more insightful, I can also create: • Boxplots (to show spread and outliers) • Time series plots (if rows represent time/order) • Correlation heatmaps • Side-by-side comparisons (e.g., y0 vs y0.1) Just tell me Prompt: What is your version information? You're chatting with ChatGPT based on the GPT-5.3 architecture. Key details: • Model: GPT-5.3 • Knowledge cutoff: August 2025 • Current date (system): March 31, 2026 If you need more technical details (capabilities, differences vs GPT-4/5, etc.), just let me know (This chat took place on Tuesday, March 31st, 2026.)

Narrative compare and contrast:

The two visulzation techniques are very different in several aspects even though they are conveying the same dataset, this is as expected though when comparing the two prompts. The one main difference is how the planned plot is able to effectivey seperate the two treatment groups so the visuzation clearly shows the changes over time. When looking at this compared to the generic plot which has no distinguishment between groups making it impossible to compare treatments. When also trying to make comparisons over time, the planned plot is much more effecitve then the generic plot which only shows static comparisons between the columns.There is also a difference in the data structure, which overall was one of the main reasons that my plan focused on wrangling and tidying the data. In the planned plot each row is one individual observation showing that the data is tidy. The original plot has the original wide format and more summarizes the columns. Overall we can conclude that the planned plot allows for much deeper and in depth analysis for treatment and it's impacts on calcium, where as the generic prompt plot allows for mere simple interpretation about averages.

#Self Reflection I have definetly learned a lot from this course, even starting back with goal setting. I think it was very helpful that we did not learn the planning process with coding examples, instead we started with the paper boats. By showing how the goal setting, needs, and steps could apply to something so simple it showed how it can be used in daily life and is definetly a critical skill I will take beyond this course. Another one of the more coding related things I took with me was using the ggplot functions especially with creating data visualtions. An example of this was the airport dataset which I am choosing to reflect on because it encorperated many aspects from the course. We got the data in an untidy form so we had to wrangle and tidy it which we learned about earlier in the semester. Then we used multiple kinds of visuzation such as a table and a graph, and were able to compare them. These summary comparisons were also a big part of the course that really helped me further analyze data and think critically.

GEN AI USAGE APPENDIX

Tool: Chat Gpt version 5.3 Date: 4-13-26 Prompt used: Using the calcium.csv dataset follow my plan to tidy and wrangle the data. The dataset is not tidy and contains calcium measurements across time for different two groups Reshape the data into a tidy format with columns for subject ID, treatment group, time (Initial, Year 1, Year 2, Year 3), and calcium measurement. Then create a ggplot2 visualization that shows how calcium levels change over time for both groups. Use a line plot and include labels. Then summarize and explain what the plot shows about differences between the two groups.

Exact Response Generated:

```r
if (FALSE) {
  library(tidyverse)
  library(readr)

  calcium <- read_csv("calcium.csv")
  glimpse(calcium)

  calcium <- calcium %>%
    mutate(subject_id = row_number())

  calcium_tidy <- calcium %>%
    pivot_longer(
      cols = -subject_id,
      names_to = "variable",
      values_to = "calcium"
    )

  calcium_tidy <- calcium_tidy %>%
    separate(variable, into = c("group", "time"), sep = "_")

  calcium_tidy <- calcium_tidy %>%
    mutate(
      time = factor(
        time,
        levels = c("Initial", "Year1", "Year2", "Year3"),
        labels = c("Initial", "Year 1", "Year 2", "Year 3")
      )
    )

  head(calcium_tidy)
```

```r
  ggplot(calcium_tidy, aes(x = time, y = calcium, group = subject_id, color = group)) +
    geom_line(alpha = 0.4) +
    stat_summary(aes(group = group), fun = mean, geom = "line", size = 1.5) +
    labs(
      title = "Calcium Levels Over Time by Treatment Group",
      x = "Time",
      y = "Calcium Measurement",
      color = "Treatment Group"
    ) +
    theme_minimal()
}
```

Summary of the Visualization

The line plot shows how calcium levels change over time for individuals in two different treatment groups. Each faint line represents an individual subject, while the thicker lines represent the average trend for each group.

Key Insights Overall Trend: Calcium levels change over time in both groups, but the direction and magnitude of change differ. Group Differences: One treatment group shows a more consistent increase (or decrease, depending on your dataset), while the other group changes more gradually or remains relatively stable. Variability: There is noticeable variation between individuals within each group, but the average lines help clarify the overall pattern. Interpretation: The separation between the group average lines suggests that the treatment may have an effect on calcium levels over time

{{< pagebreak >}}
Code Appendix

```r
#==============
# Buisest Airports Wrangling
#===============

library(tidyverse)
library(rvest)
library(janitor)
library(knitr)


wiki_url <- "https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffic"

airport_lookup <- tribble(
  ~airport, ~iata,
```

```r
  "Hartsfield-Jackson Atlanta International Airport", "ATL",
  "Frankfurt Airport", "FRA",
  "Beijing Daxing International Airport", "PKX",
  "John F. Kennedy International Airport", "JFK",
  "Miami International Airport", "MIA",
  "Hamad International Airport", "DOH"
)


airport_tables <- read_html(wiki_url) |>
  html_elements("table.wikitable") |>
  html_table(fill = TRUE)


clean_year_table <- function(table_data, year_value) {
  table_data |>
    clean_names() |>
    mutate(across(everything(), as.character)) |>
    mutate(across(everything(), ~ str_remove_all(.x, "\\[\\d+\\]"))) |>
    mutate(across(everything(), str_squish)) |>
    rename_with(~ "airport", matches("^airport$|airport")) |>
    rename_with(~ "passengers", matches("total_passengers|passengers|traffic")) |>
    select(airport, passengers) |>
    mutate(
      passengers = str_remove_all(passengers, ","),
      passengers = str_extract(passengers, "\\d+"),
      passengers = as.numeric(passengers),
      year = year_value
    ) |>
    filter(!is.na(airport), !is.na(passengers))
}


airports_tidy <- bind_rows(
  clean_year_table(airport_tables[[1]], 2025),
  clean_year_table(airport_tables[[2]], 2024),
  clean_year_table(airport_tables[[3]], 2023),
  clean_year_table(airport_tables[[4]], 2022),
  clean_year_table(airport_tables[[5]], 2021),
  clean_year_table(airport_tables[[6]], 2020)
) |>
  semi_join(airport_lookup, by = "airport") |>
```

```r
  left_join(airport_lookup, by = "airport") |>
  select(airport, iata, year, passengers) |>
  arrange(airport, year)

airports_tidy


#===========
# Buisiest Aiports Table
#===========

airports_table <- airports_tidy |>
  mutate(
    airport = case_when(
      airport == "Hartsfield-Jackson Atlanta International Airport" ~ "Atlanta International
      airport == "Beijing Daxing International Airport" ~ "Beijing Daxing International",
      airport == "John F. Kennedy International Airport" ~ "John F. Kennedy International",
      airport == "Miami International Airport" ~ "Miami International",
      airport == "Frankfurt Airport" ~ "Frankfurt",
      airport == "Hamad International Airport" ~ "Hamad International",
      TRUE ~ airport
    )
  ) |>
  pivot_wider(
    names_from = year,
    values_from = passengers
  ) |>
  arrange(airport) |>
  rename(
    `Airport Name` = airport,
    `IATA Code` = iata
  )

airports_table |>
  kable(
    caption = "Table 1: Passenger traffic for six of the busiest airports in the world, 2020
    digits = 0,
    align = c("l", "c", "r", "r", "r", "r", "r", "r"),
    booktabs = TRUE
  )

#===========
```

```r
# Buisiest Aiports Graph
#===========

airports_tidy |>
  ggplot(aes(x = year, y = passengers, color = iata, group = airport)) +
  geom_line(linewidth = 1.5) +
  geom_point(size = 4) +
  scale_x_continuous(breaks = 2020:2025) +
  scale_y_continuous(labels = function(x) format(x, scientific = FALSE))
  labs(
    title = "Passenger Traffic for Six Major Airports, 2020-2025",
    x = "Year",
    y = "Passengers",
    color = "Airport"
  )
#===========
# Monte Carlo Function
#===========

library(tidyverse)

rand_points <- function(n, x_min, x_max, y_min, y_max) {
  tibble(
    x = runif(n, min = x_min, max = x_max),
    y = runif(n, min = y_min, max = y_max)
  )
}

#===========
#Monte Carlo Plots
#===========
n_values <- c(10,100,1000,10000)

plots<- list()

for (n in n_values) {



  mc_data <- rand_points(n = n, x_min = 0, x_max = 20, y_min = 0, y_max = 0.2)

  mc_plot_data <- mc_data |>
```

```r
 mutate(
  curve_y = dchisq(x, df = 5),
  position = if_else(y <= curve_y, "On/Below Curve", "Above Curve")
  )
mc_estimate <- mc_plot_data |>
  summarize(
    prop_under = mean(position == "On/Below Curve"),
    rectangle_area = (20 - 0) * (0.2 - 0),
    estimate = prop_under * rectangle_area
  ) |>
  pull(estimate)
p <- mc_plot_data |>
  ggplot(aes(x = x, y = y, color = position)) +
  geom_point(alpha = 0.7, size = 2) +
stat_function(
  fun = dchisq,
  args = list(df = 5),
  xlim = c(0, 20),
  linewidth = 1.2,
  color = "blue"
) +
```

```r
  annotate(
    "text",
    x = 14,
    y = 0.18,
    label = paste("Estimated Integral =", round(mc_estimate, 4))
  ) +
  labs(
    title = "Monte Carlo Integration for a Chi-square Density",
    subtitle = paste("Sample Size (n) =", n),
    x = "x",
    y = "y",
    color = "Point Position"
  )
  plots[[as.character(n)]] <- p
}
plots[["10"]]
plots[["100"]]
plots[["1000"]]
plots[["10000"]]
```

```r
1 + 1
```

```
[1] 2
```

You can add options to executable code like this

```
[1] 4
```